ECE551: Digital System Design and Synthesis

Fall 2026

Verilog Basics
Structural Verilog

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Verilog Basics – Modules

- Design encapsulated in a **module**
  - Between keywords **module** and **endmodule**
  - Need a module name to refer to design
- Input and outputs are through **ports**
  - Need port name and direction
  - Can specify signal type and size

This defines the module **interface**

```
module decoder_2_to_4 (
  input [1:0] A,   // input to decode
  output [3:0] D   // one-hot output
) ;
```

Alternative port declaration style

module name *port names*

port size

port direction

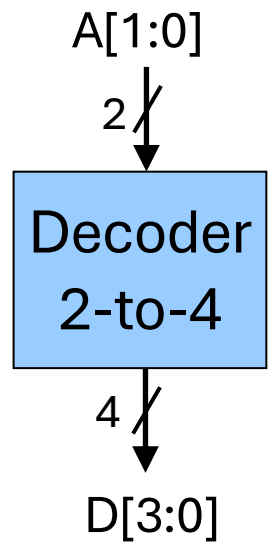
```
module decoder_2_to_4 (A, D) ;

  input [1:0] A ;   // input to decode
  output [3:0] D ;  // one-hot output

  assign D =   (A == 2'b00) ? 4'b0001 :
               (A == 2'b01) ? 4'b0010 :
               (A == 2'b10) ? 4'b0100 :
               (A == 2'b11) ? 4'b1000 ;
endmodule
```

module contents

keywords bold



Verilog Basics – Ports

- Port directions
  - **input**
  - **output**
  - **inout** (bidirectional)

- Port size
  - Scalar (single bit) – default when no size is specified
    - **input** cin;
  - Vector (multiple bit) – specify size using [<range>] (**before** signal)
    - **output** [7:0] out; // Convention is MSB to LSB (left to right)
    - **input** [0:4] in; // Legal syntax
    - **inout** [3:2] data; // Range can be between any 2 numbers, don't have to include 0

Verilog Basics – Vectors

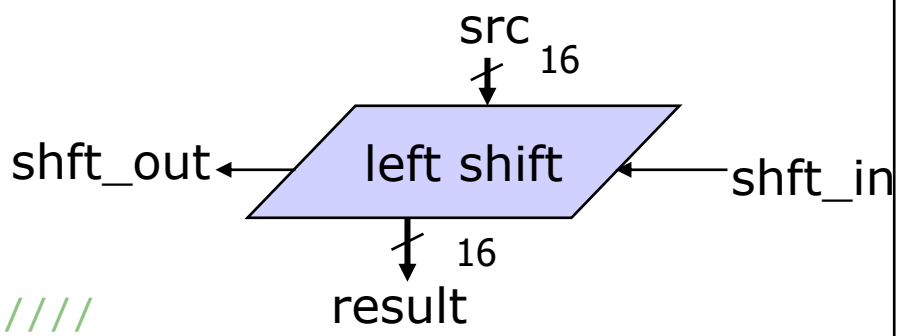
- Signal name refers to the entire vector
- Select parts of a vector using [<range>] **after** the signal name

```
module lft_shift(src,shft_in,result,shft_out);
  input [15:0] src;
  input shft_in;
  output [15:0] result;
  output shft_out;

  ////////////////////////////////////////////////////////
  // Can access 15 LSB's of src with [14:0] selector    //
  // { , } is a process of concatenating two vectors to form one //
  ////////////////////////////////////////////////////////
  assign result = {src[14:0],shft_in};

  assign shft_out = src[15];     // can access a single bit MSB with [15]

endmodule
```



Verilog Basics – Signal Types

- Signal is physically voltage on a wire
- Main synthesizable Verilog types:
  - **wire** – for signals driven by purely combinational logic
    - Needs a driver (e.g., primitive gate, module, I/O)
  - **reg** – for inferring logic in procedural (always) block
    - Retain value until a new one is assigned
    - Can be changed in simulation by assigning a new value

Verilog Basics – Signal Types

- Signal is physically voltage on a wire
- Main synthesizable Verilog types:
  - **wire** – for signals driven by purely combinational logic
  - **reg** – for inferring logic in procedural (always) block

```
module mult_accum (
   input [7:0] A_op, B_op,
   input [15:0] accum,
   output [15:0] result
);
   wire [15:0] product;

   // First multiply to form product
   assign product = A_op * B_op;    // infer HW multiplier

   // Then accumulate to form result
   assign result = accum + product;

endmodule
```

```
module mult_accum_pipelined (
   input [7:0] A_op, B_op,
   input [15:0] accum,
   output [15:0] result
);
   reg [15:0] product;

   // First multiply to form product
   always @(posedge clk)
      product <= A_op * B_op; // infer HW multiplier & FF

   // Then accumulate to form result
   assign result = accum + product;

endmodule
```

Verilog Basics – Signal Types

- Signal is physically voltage on a wire
- Main synthesizable Verilog types:
  - **wire** – for signals driven by purely combinational logic
  - **reg** – for inferring logic in procedural (always) block

```
module mult_accum (
   input [7:0] A_op, B_op,
   input [15:0] accum,
   output [15:0] result
);
   wire [15:0] product;

   // First multiply to form product
   assign product = A_op * B_op;    // infer HW multiplier

   // Then accumulate to form result
   assign result = accum + product;

endmodule
```

```
module mult_accum (
   input [7:0] A_op, B_op,
   input [15:0] accum,
   output [15:0] result
);
   reg [15:0] product;

   // First multiply to form product
   always_comb
      product = A_op * B_op; // infer HW multiplier

   // Then accumulate to form result
   assign result = accum + product;

endmodule
```

Verilog Basics – Signal Types

- Signal is physically voltage on a wire
- Main synthesizable Verilog types:
  - **wire** – for signals driven by purely combinational logic
  - **reg** – for inferring logic in procedural (always) block

```
module mult_accum (
   input [7:0] A_op, B_op,
   input [15:0] accum,
   output [15:0] result
);
   logic [15:0] product;

   // First multiply to form product
   assign product = A_op * B_op;    // infer HW multiplier

   // Then accumulate to form result
   assign result = accum + product;

endmodule
```

```
module mult_accum_pipelined (
   input [7:0] A_op, B_op,
   input [15:0] accum,
   output [15:0] result
);
   logic [15:0] product;

   // First multiply to form product
   always @(posedge clk)
      product <= A_op * B_op; // infer HW multiplier & FF

   // Then accumulate to form result
   assign result = accum + product;

endmodule
```

- SystemVerilog: **logic** – superset of reg and wire

Verilog Basics – Signal Types

- Signal is physically voltage on a wire
- Main synthesizable Verilog types:
  - **wire** – for signals driven by purely combinational logic
  - **reg** – for inferring logic in procedural (always) block

```
module mult_accum (
   input [7:0] A_op, B_op,
   input [15:0] accum,
   output [15:0] result
);
   logic [15:0] product;

   // First multiply to form product
   assign product = A_op * B_op;    // infer HW multiplier

   // Then accumulate to form result
   assign result = accum + product;

endmodule
```

```
module mult_accum_pipelined2 (
   input [7:0] A_op, B_op,
   input [15:0] accum,
   output logic [15:0] result
);
   logic [15:0] product;

   // First multiply to form product
   always @(posedge clk)
      product <= A_op * B_op; // infer HW multiplier & FF

   // Then accumulate to form result
   always @(posedge clk)
      result <= accum + product; // infer adder & FF

endmodule
```

- SystemVerilog: **logic** – superset of reg and wire

Verilog Basics – Signal Types

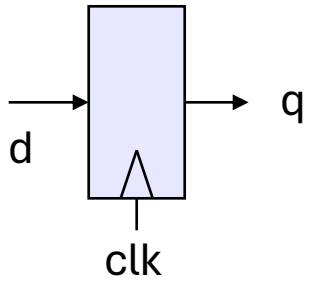
- Signal is physically voltage on a wire
- Main synthesizable Verilog types:
  - **wire** – for signals driven by purely combinational logic
  - **reg** – for inferring logic in procedural (always) block
- SystemVerilog: **logic** – superset of reg and wire

- Recommended to use logic in general
  - Special case: use wire if you have multiple drivers with different strength

Verilog Basics – Signal Value

- logic, reg, wire: 4-state data types

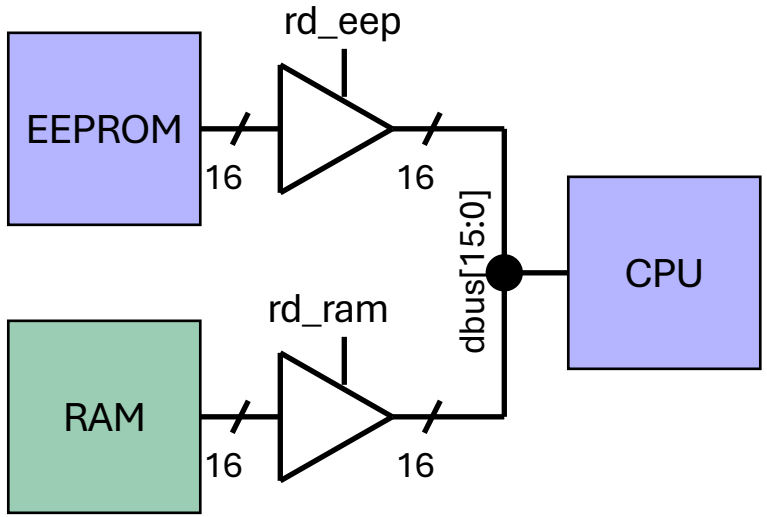
Value	Meaning
0	Logic zero, false condition
1	Logic one, true condition
x	Unknown, either uninitialized or in contention
z	High impedance. Not driven, or not connected



What value does q have prior to any positive clock edge?

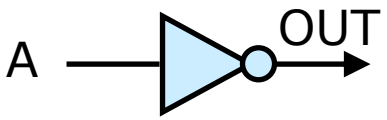
It is unknown: **x** (uninitialized)

Need high impedance for modeling of tri-state busses

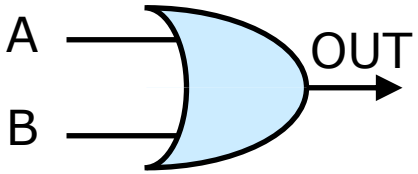


Verilog Basics – Signal Value

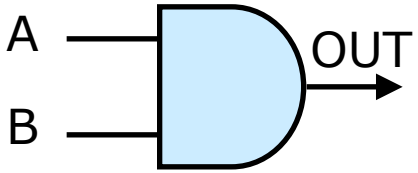
- How to 4-value logic resolve?



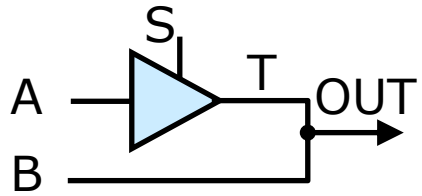
A	0	1	x	z
OUT	1	0	x	x



A	B	OUT
0	0	0
0	1	1
1	1	1
0	x	x
0	z	x
1	x	1
1	z	1



A	B	OUT
0	0	0
0	1	0
1	1	1
0	x	0
0	z	0
1	x	x
1	z	x



S	A	T	B	OUT
0	0	z	z	z
0	1	z	x	x
0	x	z	1	1
0	z	z	0	0
1	0	0	1	x
1	0	0	z	0
1	1	1	z	1
1	x	x	z	x
1	z	x	0	x

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Verilog Basics – Numbers

- General format is: <size>’<base><number>
- Specify size based on smallest needed in the design
  - If not specified, default size depends on simulator/machine
- Available bases:
  - d – decimal (default)
  - h – hexadecimal (used very often)
  - b – binary (used often for smaller numbers)
  - o – octal
- Number can have x or z characters as values
  - x = unknown
  - z = high impedance
- Supports negative numbers as well

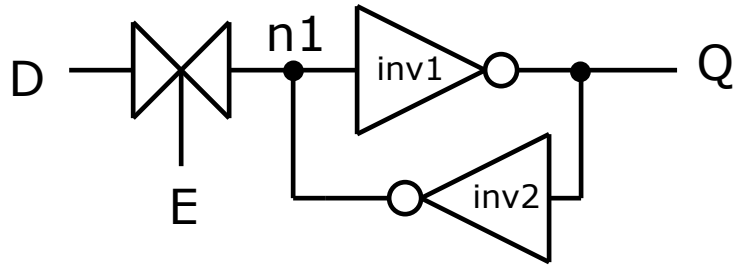
Examples:

```
4’b1101   //4-bit binary number equal to 13
10’h2E7   //10-bit # in hex equal to 2E7 (743)
10’hFE7   //10-bit # in hex equal to 3E7 instead of FE7
12’h13x   //12-bit hex # with lower 4 bits unknown
-16’h3A   //16-bit # with -3A in hex (FFC6 in 2’s comp)
```

Verilog Basics – Signal Strength

- Concept of drive strength supported to model certain CMOS circuits that utilize contention
- Not used often in general design

Strength Level	Meaning
supply	Strongest (like a supply VDD or VSS)
strong	Strong (default if not specified)
pull	Medium strength (like a pullup device)
weak	Weak, like a keeper (sustainer, leaker)
highz	High impedance




```
not                    inv1 (Q,n1)
not (weak0,weak1) inv2 (n1,Q)
```

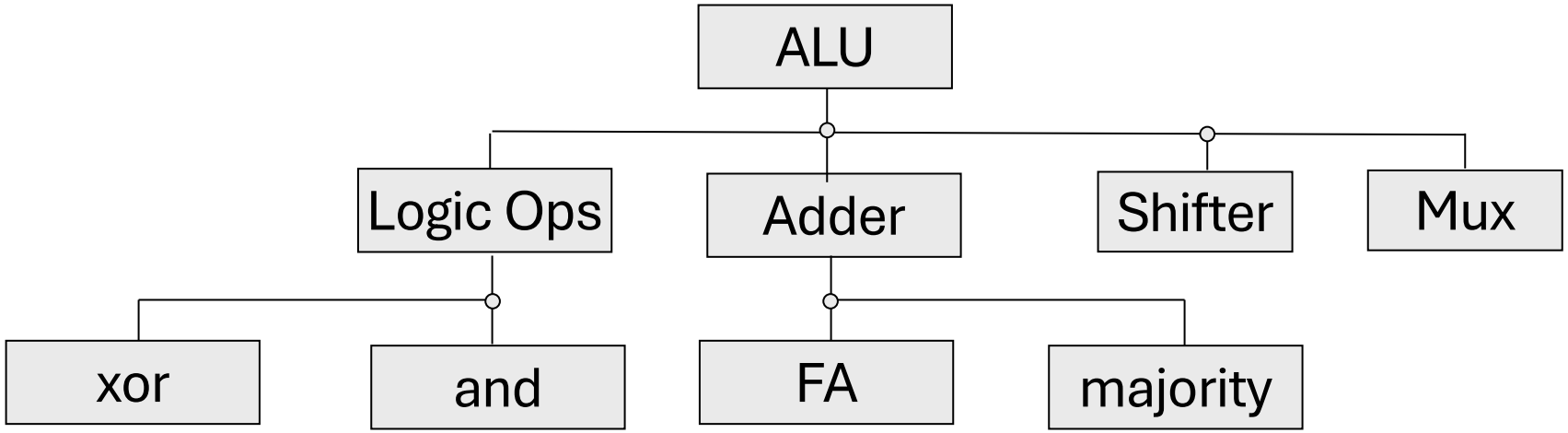
Module Coding Styles

- Modules can be specified in different ways
  - Structural – connect primitives and modules
  - Dataflow – use continuous assignments
  - Behavioral – use initial and always blocks

- A single module can use more than one method

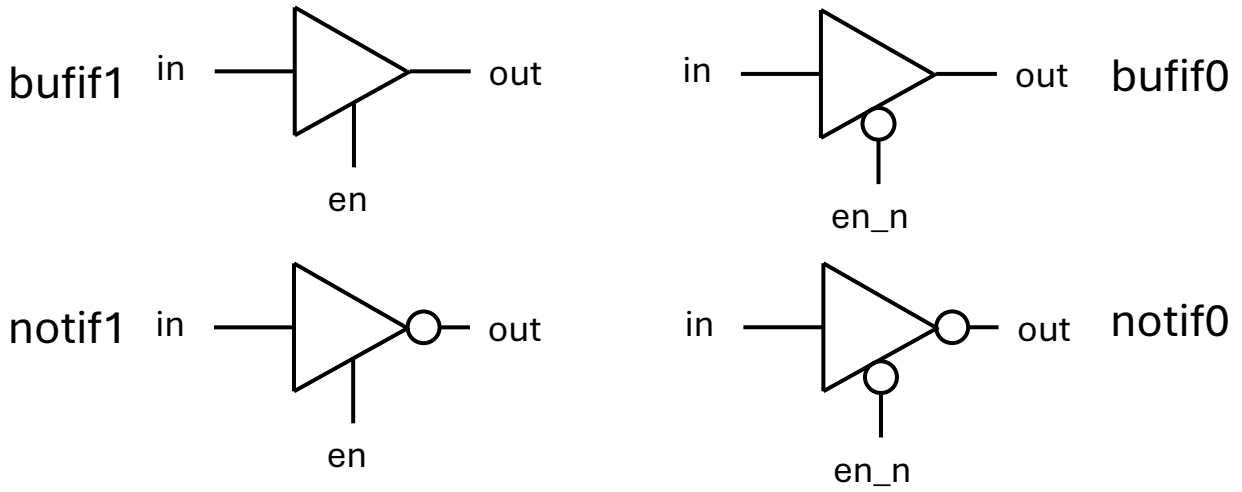
Structural Verilog

- A schematic in text form (i.e., a netlist)
- Build up a circuit from gates and modules
  - Gates are primitives (part of the language)
  - Modules are user-defined
- Build up hierarchy from smaller pieces when we design
  - Architecture: typically top-down
  - Design & verification: typically bottom up



Built-in Gate Primitives

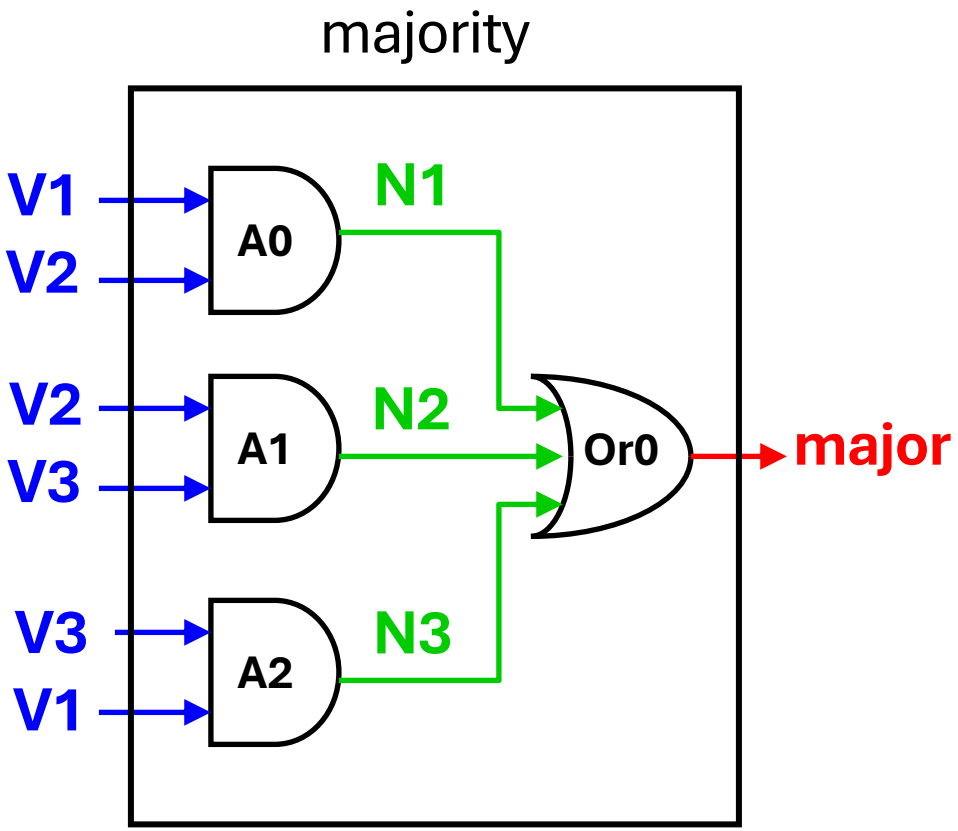
- Basic logic gates:
  - and, or, xor, nand, nor, xnor
  - buf, not
  - bufif0, bufif1, notif0, notif1



- Port order: output(s), then input(s), then control if it exists
- Can optionally specify instance name , delay, strength

```
and (Z,A,B,C);                          // 3-input AND gate
and N25 (Z,A,B,C);                      // name specified
and #10 (Z,A,B,X),  (X,C,D,E);          // delay specified, 2 gates
and (weak0,weak1) #5 N30 (Z,A,B);       // name, strength and delay specified
```

Structural Verilog Example




```
module majority (major, V1, V2, V3) ;

  output major ;
  input V1, V2, V3 ;

  logic N1, N2, N3;

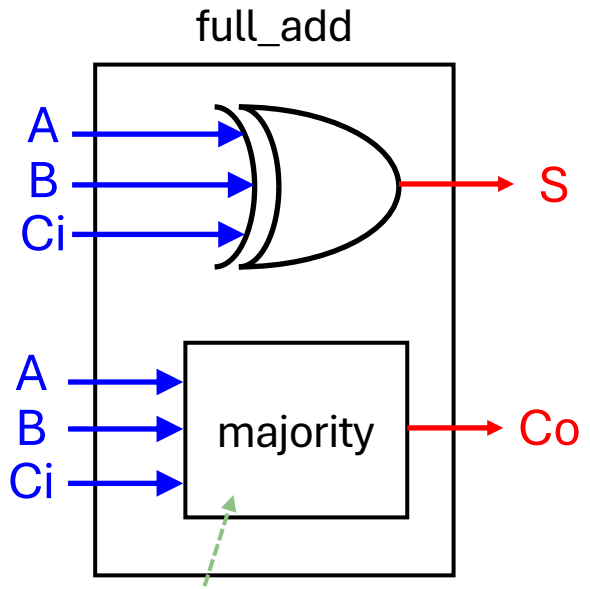
  and A0 (N1, V1, V2),
      A1 (N2, V2, V3),
      A2 (N3, V3, V1);

  or Or0 (major, N1, N2, N3);

endmodule
```

Structural Verilog Example

- Can build modules from other modules
  - Instance name is required when instantiating user-defined modules




```
module majority (major, V1, V2, V3) ;
```

```
module full_add (A, B, Ci, S, Co) ;

  input A, B, Ci ;
  output S, Co ;

  // Sum is XOR of inputs
  xor iSUM (S, A, B, Ci);

  // Co is the majority of inputs
  majority iCO(.V1(A), .V2(B), .V3(Ci), .major(Co));

endmodule
```

Connect by reference order

Connect by name

Alternative is to connect by order…

```
majority iCO(Co, A, B, Ci);
```

Port Connections

- Positional / connect by order
  - Ok for designs with very few ports & interchangeable input ports (e.g., and/or/xor)
- Explicit / connect by name
  - Helps avoid connection errors, no need to remember port order, easier to read
  - `.<port name>(<signal name>)`
- Empty port connections
  - Input: high impedance
  - Output: not used

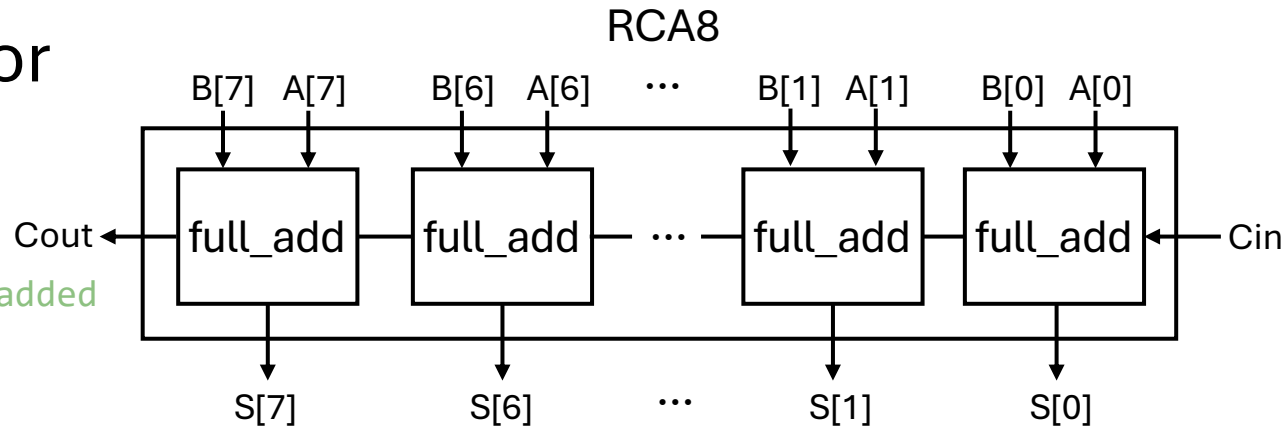
Example: `module full_add(A,B,Ci,S,Co);`

```
full_add iadd(.A(X),.B(Y),.S(Z),.Co(ov));     //Ci is high impedance
full_add iadd(.A(X),.B(Y),.Ci(1'b0).S(Z), ); //Output Co unused

full_add iadd(.A(X),.B(Y),.Ci(1'b0).S(Z), .Co()); //Better code
```

Structural Verilog Example

- Can instantiate modules as a vector




```
module RCA8 (
  input [7:0]  A,B,          // two 8-bit vectors to be added
  input        Cin,          // An optional carry in bit
  output [7:0] S,            // 8-bit sum
  output       Cout          // carry out
);

  // Co of lesser significant bit form Cin of more significant bit
  logic [7:0] Carries;

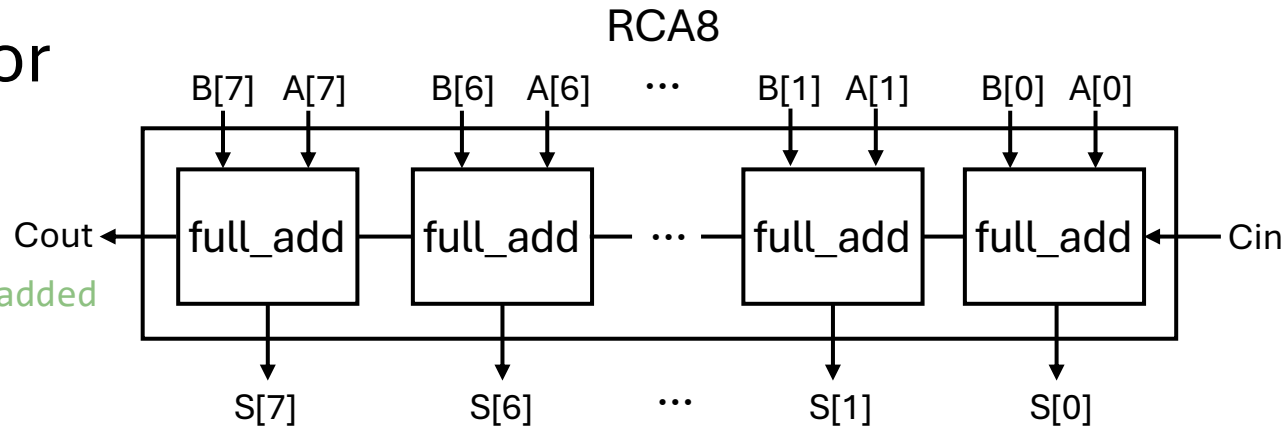
  // Implement Full Adder as structural Verilog
  full_add add0(.A(A[0]), .B(B[0]), .Cin(Cin),         .S(S[0]), .Cout(Carries[0]));
  full_add add1(.A(A[1]), .B(B[1]), .Cin(Carries[0]), .S(S[1]), .Cout(Carries[1]));
  full_add add2(.A(A[2]), .B(B[2]), .Cin(Carries[1]), .S(S[2]), .Cout(Carries[2]));
  full_add add3(.A(A[3]), .B(B[3]), .Cin(Carries[2]), .S(S[3]), .Cout(Carries[3]));
  ...
  full_add add7(.A(A[7]), .B(B[7]), .Cin(Carries[6]), .S(S[7]), .Cout(Carries[7]));

  buf iBF(Cout, Carries[7]);

endmodule
```

Structural Verilog Example

- Can instantiate modules as a vector




```
module RCA8 (
  input [7:0]  A,B,          // two 8-bit vectors to be added
  input        Cin,          // An optional carry in bit
  output [7:0] S,            // 8-bit sum
  output       Cout          // carry out
);

  // Co of lesser significant bit form Cin of more significant bit
  logic [7:0] Carries;

  // Implement Full Adder as structural Verilog
  full_add iFA[7:0](.A(A), .B(B), .Cin({Carries[6:0],Cin}), .S(S), .Cout(Carries));

  buf iBF(Cout, Carries[7]);

endmodule
```

Why Know Structural Verilog?

- DUT code will mostly be dataflow or behavioral
- Testbenches primarily in behavioral

- What needs structural Verilog?
  - Building hierarchy (instantiating blocks to form higher level functional blocks)
  - Synthesis tools output structural Verilog (gate-level netlist)
    → you need to be able to read this output

